Module 7: WMATA Accessibility

JHU EP 605.206 - Introduction to Programming Using Python

Introduction

The Washington Metropolitan Area Transit Authority (WMATA) provides metro station access to riders with disabilities by providing elevators and escalators to under- and above-ground transportation. Occasionally, these services become unavailable due to maintenance and mechanical issues; we'll call such an occurrence an "incident". These incidents can have major impacts on riders with mobility issues. The WMATA makes incident data publicly available through a set of web API's. We will use these API's, and design one of our own, to enable riders with disabilities to more easily navigate the DC metro system.

This Assignment contains 2 parts:

1. Use WMATA/MapBox API's to create a map with markers denoting metro stations with degraded elevator/escalator service
2. Design/document a new "wrapper" API to improve the WMATA Incidents API

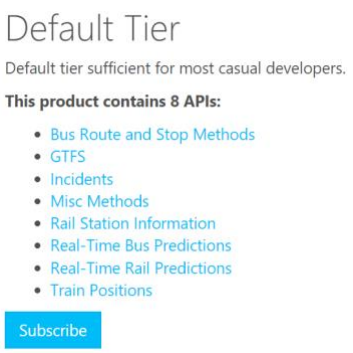
Part 1 of the programming portion of the Assignment will work a little differently this week. We'll provide you the core piece of code (wmata.py) and you will work to fill it in based on the comments. We've provided ~90 lines of code and you will be responsible for writing an additional 30-40.

For Part 2 you will use Swagger Editor design an API and produce the human-readable documentation.

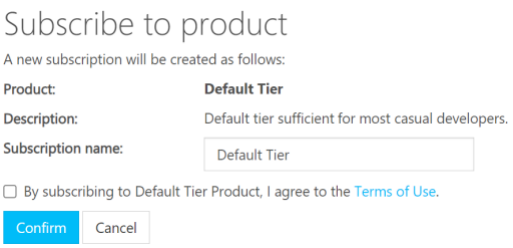
Skills: REST API's, Python requests, OpenAPI/Swagger, web services, creating files using binary data

Setting Up a WMATA API Key (Required)

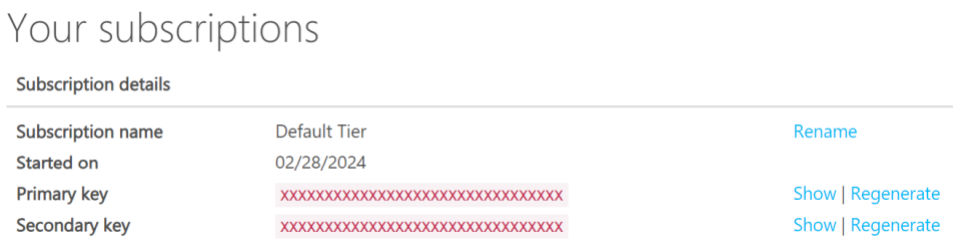
1. Create an account to register your personal WMATA API key
    a. **NOTE**: You may try using the demo key but we've experienced many difficulties in doing so!
2. Verify your email address
3. Select "**Default Tier**"
4. Click "**Subscribe**"



5. Agree to the Terms of Use and click "**Confirm**"

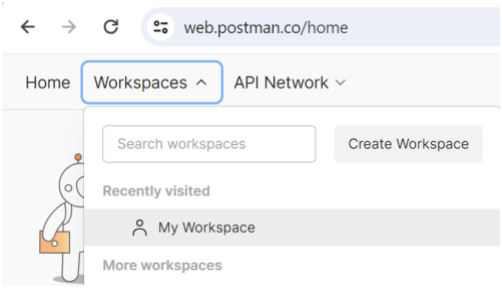


6. Retrieve your API key by clicking "**Show**" on the "**Primary key**"

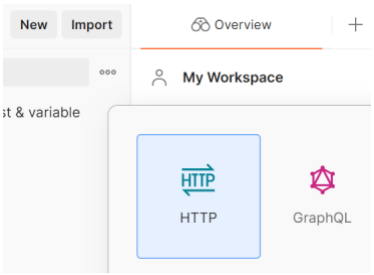


Exploring the WMATA API with Postman Web (Recommended)

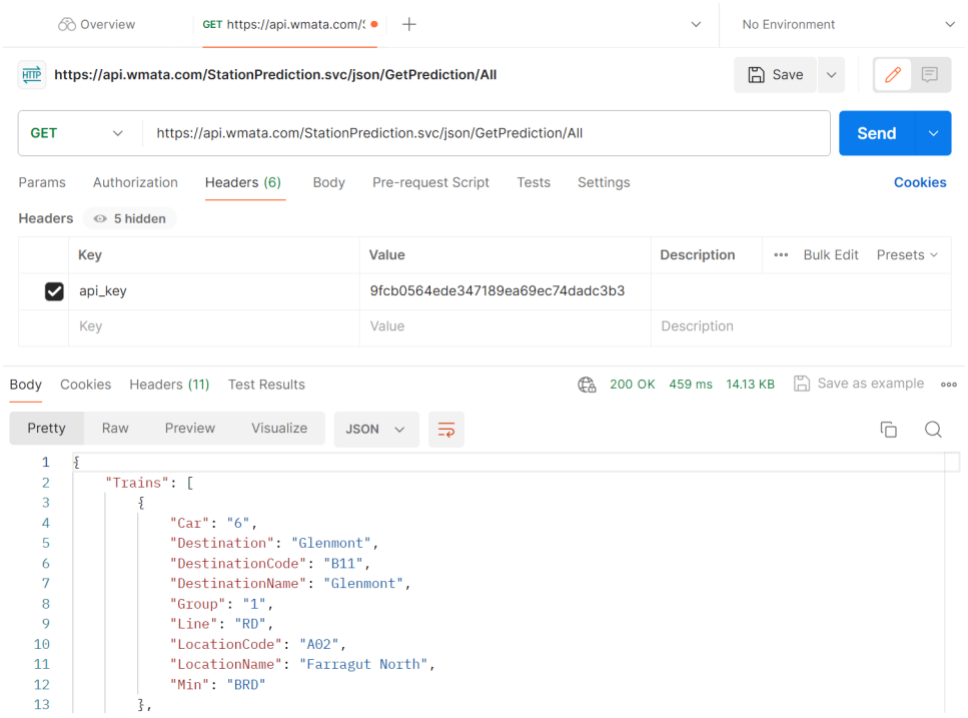
1. Register for a free Postman account
2. Go to “My Workspace”



3. Create a new HTTP request



4. Create a test request to the WMATA Incidents API using your WMATA API key



5. Review the results (JSON data) returned in the body of the response

Programming Assignment Part 1 - Consuming a REST API

In Part 1 of the Programming Assignment we will use WMATA + MapBox API's to generate a static map image containing markers at each metro station currently experience an elevator/escalator outage. Part 1 contains 3 steps: getting a list of incidents, getting location information for the stations where these incidents have occurred, and creating a map (image) displaying the affected locations.

Step 1: Get a List of Elevator/Escalator Incidents from the WMATA API

1. Download **wmata.py** from Canvas
    a. You'll fill-in this code as you progress through Programming Assignment Part 1
2. Replace the <YOUR_WMATA_API_KEY> with your personal WMATA API key
3. Complete the **`get_station_incidents()`** function (and call it from __main__)
    a. Use the Python **`requests`** library to query the WMATA Incidents API
        i. This will return a list of ElevatorIncident objects
    b. Create a Python set of unique station codes from the ElevatorIncident objects
    c. Return the result of (b) to the caller
4. Call the get_station_incidents() function from within __main__ and store the return value
    a. Print the number of metro stations experiencing accessibility issues:
        i. Format: "X stations are currently experiencing accessibility outages."
            1. X is the number of stations returned from (4)
5. The set of unique station codes from (4) will be used in the next step

Step 2: Retrieve Station Location Information (longitude, latitude) from the WMATA API

In Step 2 of the Assignment you will take the station codes from Step 1 and look up their geographical coordinates (lon/lat) in a different part of the WMATA API: Rail Station Information. This pattern of linking data together by calling various API's is very common (though a little bit cumbersome). Tools like GraphQL solve this problem but are outside the scope of this course.

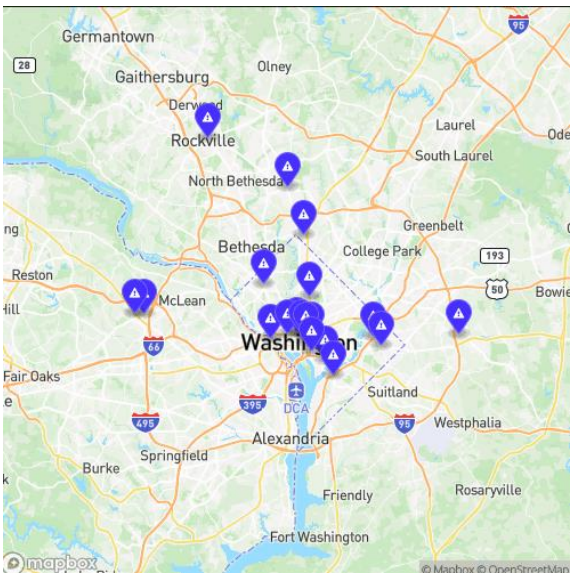
1. Inside of __main__:
    a. Create an empty to list to keep track of lon/lat tuples for each station's location
    b. For each unique station code in the result from Step 1
        i. Call the **`get_station_info()`** function which should:
            1. Use **`requests`** to query the WMATA Rail Station Information API
            2. Return the JSON-formatted response
        ii. Print the name of each station to the console
            1. Capture a screenshot of the output to be submitted later
        iii. For the first 20 or less entries from (a):
            1. Create a tuple of the form (lon, lat) and add it to the list from (a)
                a. The 20 (or less) tuples are due to a limitation of the MapBox API
    c. The result should be a list of (lon, lat) tuples in the following format:
        i. `[(lon1, lat1), (lon2, lat2), ..., (lonX, latX)]`
        ii. This list of lon/lat pairs will be used in Step 3
    d. A sample output after Step 1 (4.a) and Step 2 (1.b.ii) have been completed is shown below (though your number of stations, and the names of those stations, will most likely vary):

```
24 stations are currently experiencing accessibility outages.
Metro Center
West Hyattsville
Columbia Heights
Tenleytown-AU
Medical Center
Farragut North
Metro Center
Minnesota Ave
Pentagon
Tysons
Benning Road
Fort Totten
Downtown Largo
Spring Hill
Grosvenor-Strathmore
Braddock Road
Silver Spring
McPherson Square
Judiciary Square
New Carrollton
Dupont Circle
Georgia Ave-Petworth
Farragut West
Dunn Loring-Merrifield
```

Step 3: Creating a Static Map Image using the MapBox API

In Step 3 you'll use MapBox's Static Images API to plot the incident locations obtained in Step 2 and retrieve an image file of the resulting map.

1. **MapBox Setup**: MapBox requires an API key for use (called a token or access token) so you'll need to create a MapBox account and create an access token. Please note that while a credit card is required to provision an access token, the Static Images API allows up to 50,000 API calls at no-cost as part of it's free tier. For reference, we incurred 14 calls to the API throughout our solution development and testing. **You shoud delete your MapBox account at the end of this Assignment.**
2. **Add MapBox API Key**: Add your API key to wmata.py to be able to make Static Images API calls
3. **Retrieve Static Map**: When you completed Part 2, the final thing you did in **`main()`** was to produce a list of lon/lat pairs (tuples) for each station where there was an elevator/escalator incident in the following format: [(lon1, lat1), (lon2, lat2), ..., ()]. We will now send this list of lon/lat pairs to the Static Images API to produce a map image. Inside of **`main()`**:
    a. Convert the list of lon/lat pairs into a URL-encoded GeoJSON blob using the provided **`encode_geojson()`** function. We'll handle all of the hard work for creating the GeoJSON portion of the MapBox URL to support the placement of multiple markers on an image and return it to you.
    b. Use the result from (a) to call **`get_static_map()`**. Within this function you must:
        i. Use **`requests`** to perform a GET request to the provided **`static_map_url`**
        ii. If the response code is 200, write the binary content from the response (image is returned as a collection of raw bytes from the API) to a file named "**map.png**".
        iii. Otherwise, print the message "**Error returned from MapBox API**"
    c. You should now have a **map.png** image in your local directory. Save this image to be submitted later. Here is a screenshot of the map generated by our solution for reference (yours will likely look a little different, but mostly similar):



Programming Assignment Part 2 - Designing & Documenting a REST API

For Part 2 of this Assignment you will design and document, but not implement, a "wrapper" API based on the WMATA Incidents API. A "wrapper" is an API that has a dependency on another API. A shortcoming with the WMATA data is that the API only allows you to poll for all incidents but not filter them by type (elevator vs. escalator). In this step you will design/document your own API to wrap the WMATA Incidents API to fix this issue. In a later Assignment you'll implement the fix by publishing your own API.

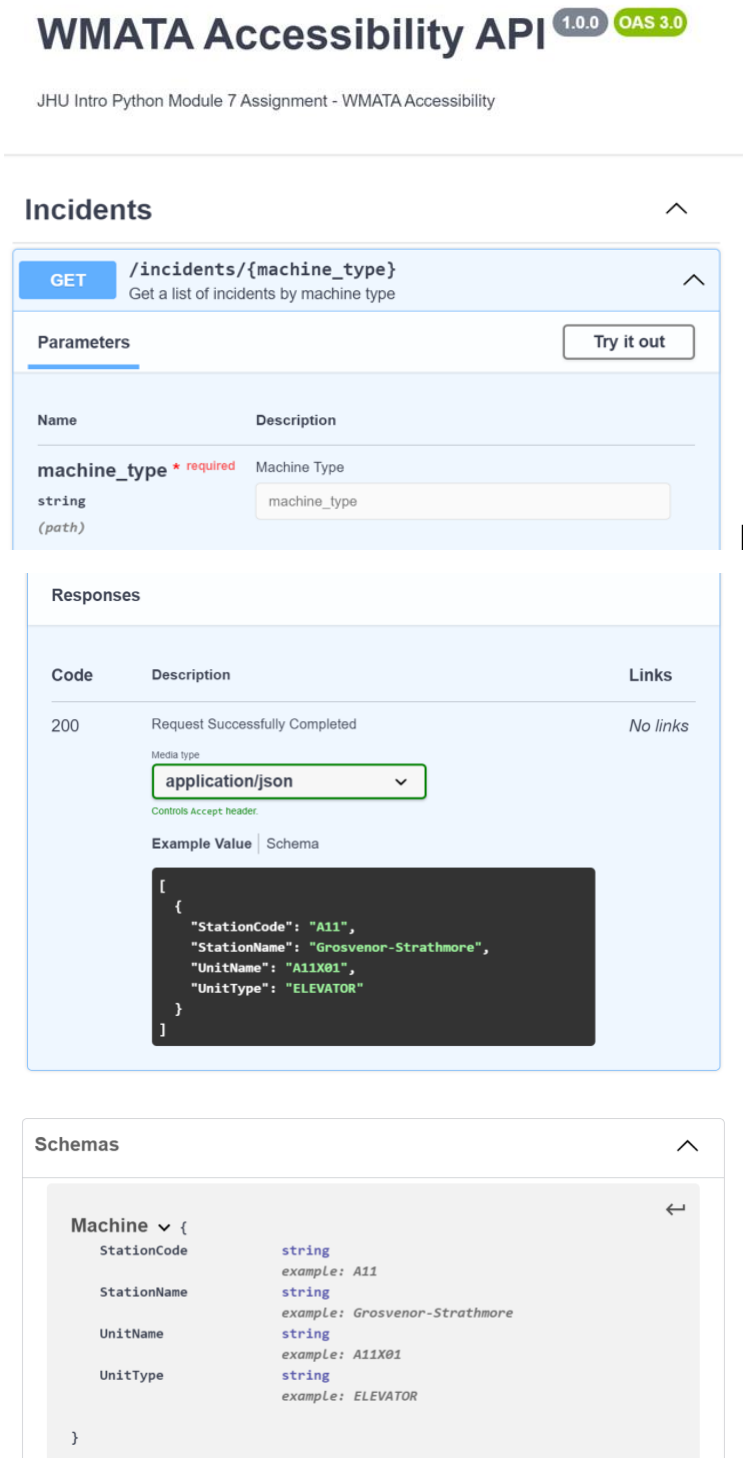
Use the Swagger Editor (online or download), or a free SwaggerHub account, to create a YAML file to document an API that adheres to the criteria below. You may find the Swagger Editor documentation, of their famous PetStore example helpful in getting your API documented. Hint: you can start by pasting the PetStore YAML definition into the Swagger Editor to see what it produces and modify it to fit your needs from there.

Your API must have a(n):

- OpenAPI version 3.0.3
- Title: "WMATA Accessibility API"
- Description: "JHU Intro Python Module 7 Assignment - WMATA Accessibility"
- Version: 1.0.0
- API endpoint path named "**incidents**" that accepts GET requests
    - Tags: "Incidents"
    - Summary: "Get a list of machine incidents by machine type"
    - A single, required path parameter named "**machine_type**"
        - Name: machine_type
        - In: path
        - Description: "Machine Type"
        - Required: true
        - Schema
            - Type: string
    - Response Code: 200
        - Description: "Request Successfully Completed"
        - Returns: application/json content
            - Schema: an array of items referencing a schema named "Machine"
- Schema component named "Machine" that is of type "object" with the following properties:
    - StationCode (string), example: "A11"
    - StationName (string), example: "Grosvenor-Strathmore"
    - UnitName (string), example: "A11X01"
    - UnitType (string), example: "ELEVATOR"

The API documentation generated from your OpenAPI YAML configuration should match the Sample API Documentation from our solution (pictured in the next section). Once your YAML configuration file is complete, save it as **incidents.yaml**; this will be submitted later.

Sample Output: API Documentation



WMATA Accessibility API 1.0.0 OAS 3.0

JHU Intro Python Module 7 Assignment - WMATA Accessibility

Incidents

GET /incidents/{machine_type}
Get a list of incidents by machine type

Parameters | Try it out

Name	Description
machine_type * required string (path)	Machine Type machine_type

Responses

Code	Description	Links
200	Request Successfully Completed Media type application/json Controls Accept header. Example Value | Schema	No links

```
[
  {
    "StationCode": "A11",
    "StationName": "Grosvenor-Strathmore",
    "UnitName": "A11X01",
    "UnitType": "ELEVATOR"
  }
]
```

Schemas

Machine {

StationCode	string	example: A11
StationName	string	example: Grosvenor-Strathmore
UnitName	string	example: A11X01
UnitType	string	example: ELEVATOR

}

Restrictions

As a reminder, all out-of-scope concepts as outlined in the Syllabus are not permitted. If you'd like to make use of additional Python features outside of the Reading and Video Lectures, please send us a note on Teams so we can weigh-in with a decision.

Concepts that are **not permitted** for use on this Assignment include, but are not limited to, the following:

- enumerate(), map(), join(), zip(), pop(), sorted(), format(), extend(), isinstance(), typing()
- List/dictionary/set comprehensions
- Generator expressions
- Exceptions, try/except, raise
- Dummy variables, global variables, global
- Any imported libraries (with the exception of those outlined in the assignment or provided in starter code)
- Argument unpacking
- PIL, bytes I/O

If you'd like to make use of additional Python features outside of the Reading and Video Lectures, please send us a note on Teams so we can weigh-in with a decision.

Deliverables

readme.txt

So-called "read me" files are a common way for developers to leave high-level notes about their applications. Here's an example of a README file for the Apache Spark project. They usually contain details about required software versions, installation instructions, contact information, etc. For our purposes, your readme.txt file will be a way for you to describe the approach you took to complete the assignment so that, in the event you may not quite get your solution working correctly, we can still award credit based on what you were trying to do. Think of it as the verbalization of what your code does (or is supposed to do). Your readme.txt file should contain the following:

1. **Name**: Your name and JHED ID
2. **Module Info**: The Module name/number along with the title of the assignment and its due date
3. **Approach**: a detailed description of the approach you implemented to solving the assignment. Be as specific as possible. If you are sorting a list of 2D points in a plane, describe the class you used to represent a point, the data structures you used to store them, and the algorithm you used to sort them, for example. The more descriptive you are, the more credit we can award in the event your solution doesn't fully work.
4. **Known Bugs**: describe the areas, if any, where your code has any known bugs. If you're asked to write a function to do a computation but you know your function returns an incorrect result, this should be noted here. Please also state how you would go about fixing the bug. If your code produces results correctly you do not have to include this section.
5. **Citations**: Please include informal citations (links, titles of articles/websites, etc.) for **any** external resources you referenced when completing the assignment.

Please submit your **wmata.py** source code file along with a Word/PDF file named JHEDID_mod7.pdf (ex: jkovba1_mod7.pdf) containing the 3+ screenshots of your outputs. Please do not ZIP your files together.

Recap:

1. readme.txt
2. wmata.py
3. incidents.yaml
4. map.png (from Part 1, Step 3)
5. A Word or PDF file named 'JHEDID_mod7' (ex: jkovba1_mod7.pdf/.doc) containing the following:
    a. 2 (or more) screenshots of your outputs:
        i. **One** screenshot of the output from **wmata.py** (the static map)
        ii. **One** screenshot of your intermediate result from Step 2
        iii. **One (or more)** screenshots displaying the generated web API documentation

Please let us know if you have any questions via Teams or email!

Text

Grading

We wanted to provide a few notes on grading to help you be as successful as possible on the assignments:

1. Following instructions closely is an important part of good software development practices. Even small details like a file in the wrong format can negatively impact software interoperability. For this reason, **we ask that you please follow the instructions as closely as possible. This includes matching the output format of the Sample Solutions exactly, submitting the required files in the correct formats, etc.**
2. The Readings + Video Lectures provide 90%+ of the skills necessary to complete this Assignment. While we encourage experimentation, we ask that you focus on the use of what we learned in this Module, and previous Modules, to develop your solution. **If you'd like to try other techniques, we ask that you please practice them separately and do not submit them as your assignment solution.**
3. Because the Readings + Video Lectures constitute a large majority of skills required to complete this assignment, we ask that you use external references as judiciously as possible. That is, **you may refer to external resources as needed, but such resources should be (informally) cited in your README and code borrowed from these resources must not be copy/pasted**.
4. **Unless otherwise specified, you do not need to worry about any error-checking or input sanitization**. We'll learn techniques for detecting and correcting errors later in the course and explicitly state when it's required.
5. **The output of your solution should match that of the Sample Outputs exactly**. It is important not just to have a functional solution but also one that adheres to the exact formatting specifications. For example, if another piece of code was expecting your results in the specified format but you added an extra word, or even an extra space, it could potentially break that code. Producing the proper output is also a good measure of your understanding in how to manipulate and display information that your code produces.

   Note: the format of the output is different than how the formatted output is displayed on the screen. Depending on your screen size, the width of your IDE window, etc., the positioning/wrapping of your output text may appear slightly differently, though still in the same format, as ours.